Deadlocks involve resources, which can be...
hardware device – tape drive, printer, memory, CPU
piece of info – variable, file, semaphore, DB record
Preemptable Resources
can be taken away from process, with no ill effects
sharable resources; CPU, memory
Non-Preemptable Resources
cannot be taken away from a process, w/o causing problems
non sharable resources; if process is burning a CD ROM, taking it back & giving to other process, will result in garbled CD
In general, DL involves Non-Preemptable resources

explain

Let's break this down **step by step**, in simple operating-system terms.

What is a Deadlock?

A **deadlock** occurs when a set of processes are **permanently blocked** because each process is **waiting for a resource** that is **held by another process** in the set.

➡️ No process can proceed, and the system makes **no progress**.

What are Resources?

A **resource** is anything a process needs to execute.

1. Hardware Resources

These are physical components of the computer:

- **Tape drive**

- **Printer**
- **Memory**
- **CPU**

2. Information Resources

These are logical or software-level items:

- **Variables**
- **Files**
- **Semaphores**
- **Database records**

Processes must **request**, **use**, and **release** these resources.

Types of Resources in Deadlocks

1. Preemptable Resources

Definition:
A **preemptable resource** can be **taken away from a process** without causing any harm or incorrect behavior.

Examples:

- **CPU** → OS can interrupt a running process and give CPU to another process
- **Memory (virtual memory)** → pages can be swapped out
- **Sharable resources**

Key Idea:
✔️ If a resource can be safely reassigned, **deadlock can be avoided**

➡️ **Deadlocks usually do NOT involve preemptable resources**

---

2. Non-Preemptable Resources

Definition:
A **non-preemptable resource cannot be taken away** from a process without causing problems.

Examples:

- **Printer** (mid-print)
- **Tape drive**
- **CD/DVD burner**
- **Database record locked for update**

Example Explanation:
If a process is **burning a CD-ROM** and the OS:

- Stops it midway
- Gives the CD drive to another process

➡️ The CD will be **corrupted / garbled**

Key Idea:
❌ Resource cannot be safely reassigned
✔️ Process must **finish using it first**

Why Deadlocks Usually Involve Non-Preemptable Resources

Deadlocks arise because:

- A process **holds** a non-preemptable resource
- While **waiting** for another non-preemptable resource
- That is held by another waiting process

Since the OS **cannot forcibly take** the resource back:

- Each process keeps waiting forever
- Deadlock occurs

Simple Deadlock Example

Process	Holds	Waiting For
P1	Printer	Tape Drive
P2	Tape Drive	Printer

➡️ Both resources are **non-preemptable**

➡️ Neither process can proceed

➡️ **Deadlock**

Summary

- **Deadlock** = circular waiting for resources
- **Resources** can be:
  - Hardware (CPU, printer)
  - Information (file, semaphore, DB record)
- **Preemptable resources**
  - Can be taken away safely
  - Rarely cause deadlocks
- **Non-preemptable resources**
  - Cannot be taken away safely
  - **Main cause of deadlocks**

Key Takeaway:

> **Deadlocks generally involve non-preemptable resources because the OS cannot forcefully reclaim them without causing errors.**

If you want, I can also explain:

- Deadlock conditions (Coffman conditions)
- Deadlock prevention vs avoidance
- Banker's Algorithm

what is mutual exclusion\

Mutual Exclusion (Mutex)

Mutual exclusion is a property that ensures **only one process or thread at a time** can access a **shared resource or critical section**.

Why Mutual Exclusion Is Needed

When multiple processes run at the same time and **share data or resources**, problems can occur such as:

- **Race conditions**
- **Data inconsistency**
- **Incorrect results**

Mutual exclusion **prevents these problems** by controlling access.

Critical Section

A **critical section** is the part of a program where:

- Shared data or resources are accessed
- Only **one process** is allowed to execute at a time

➡️ Mutual exclusion guarantees **only one process is inside the critical section**.

Simple Example

Imagine a **printer** shared by two processes:

- Process P1 starts printing
- Process P2 also wants to print

Without mutual exclusion:

- Print jobs get mixed → **garbled output**

With mutual exclusion:

- P1 prints completely
- Then P2 prints

How Mutual Exclusion Is Achieved

Common mechanisms:

- **Mutex locks**

- **Semaphores**
- **Monitors**
- **Binary semaphores**

Example (conceptual):

text Copy code

```
lock(mutex)
   critical section
unlock(mutex)
```

Mutual Exclusion in Deadlocks

Mutual exclusion is **one of the four necessary conditions for deadlock**:

1. **Mutual Exclusion –** resource cannot be shared
2. **Hold and Wait**
3. **No Preemption**
4. **Circular Wait**

➡️ If mutual exclusion is removed, **deadlock cannot occur**.

Key Points Summary

- Mutual exclusion means **one process at a time**
- Protects **shared resources**
- Prevents **race conditions**
- Required for **deadlock to happen**

In one line:

> **Mutual exclusion ensures exclusive access to shared resources so that only one process can use them at a time.**

If you want, I can also explain this with **real-life examples**, **diagrams**, or relate it to **semaphores and locks**.

Binding of instructions & data to memory addresses can be done at any of the following stages
Compile time : if memory location known a priori, absolute code can be generated at compile time
must recompile code if starting location changes
Load time : if memory location not known at compile time, relocatable code is generated – binding done at

moved from one memory segment to another, address binding delayed till run time
need hardware support for run time address mapping (e.g, relocation register)

Good topic — this is **memory management**, and your notes are basically right.
Let's **understand it instead of memorizing** 👍

First: What does *binding* mean?

Binding of instructions and data to memory addresses means:

> Deciding **where in main memory** a program's instructions and variables will be placed.

So the question is **WHEN** do we decide the actual memory addresses?

1️⃣ Compile-Time Binding

Idea

If the **exact memory location is known in advance**, the compiler can generate **absolute addresses**.

How it works

- Compiler assumes:
  "My program will start at memory address X"
- It hardcodes all addresses into the program

Example

text Copy code

```
Instruction at address 200
Variable at address 350
```

Limitation

❌ If the program is loaded at a **different memory location**, it **will not work**

➡️ You must **recompile** the program

Summary

- Binding happens **during compilation**
- Uses **absolute code**
- Very inflexible

2️⃣ Load-Time Binding

Idea

Memory location is **not known at compile time**, but **is known when the program is loaded**

How it works

- Compiler generates **relocatable code**
- Final address calculation is done by the **loader**

Example

Compiler says:

text Copy code

```
Variable is at offset +120
```

Loader says:

text Copy code

```
Program starts at 1000
Final address = 1120
```

Key Point

✔️ Program can be loaded at **any memory location**

❌ Once loaded, it **cannot move**

Summary

- Binding happens **when program is loaded**
- Uses **relocatable code**
- More flexible than compile-time binding

3️⃣ Execution-Time (Run-Time) Binding

Idea

The process **can move in memory while running**

So:

- Addresses are **not fixed** at compile time
- Not fixed at load time
- Bound **during execution**

How it works

- CPU generates **logical (virtual) addresses**
- Hardware translates them to **physical addresses**
- Uses a **relocation register / MMU**

Example

text Copy code

```
Physical Address = Logical Address + Relocation Register
```

If process moves:

- OS updates the relocation register

- Program keeps running normally

Requirement

✔ **Hardware support required**

- MMU
- Relocation register

Summary

- Binding happens **during execution**
- Maximum flexibility
- Used in modern OS

Quick Comparison Table

Binding Time	Address Type	Can Move While Running?	Hardware Needed
Compile Time	Absolute	❌ No	❌ No
Load Time	Relocatable	❌ No	❌ No
Execution Time	Logical → Physical	✅ Yes	✅ Yes

One-Line Memory Trick 🧠

Compile = Fixed forever
Load = Fixed after loading
Execute = Flexible anytime

Quick check (answer in one line 👇):

Why does **execution-time binding** require **hardware support**?

MMU – hardware device for run time address mapping
"map logical to physical address at run time"
Generalization of Base register – Relocation Register
value in relocation register, added to every address generated by user process, at the time it is sent to memory
User programs deal with logical addresses, and never see real physical addresses
when reference is made to some location in memory binding is done at run time

Great — you're describing the **MMU and run-time address binding**. Let's **tighten and clarify** it so it's exam-ready and conceptually clear.

What is the MMU?

MMU (Memory Management Unit) is a **hardware device** that performs **run-time address mapping**.

👉 Its job is to:

> **Convert logical addresses (used by programs) into physical addresses (used by memory).**

Logical vs Physical Address

- **Logical address**
  - Generated by the **CPU**
  - Used by **user programs**
  - Also called **virtual address**
- **Physical address**
  - Actual address in **main memory (RAM)**
  - Never seen directly by user programs

✔ User programs **only work with logical addresses**

MMU as a Generalization of Base (Relocation) Register

The MMU works like an advanced **base/relocation register**.

Relocation Register:

- Stores the **starting physical address** of the process in memory

Address Mapping Formula:

text Copy code

```
Physical Address = Logical Address + Relocation Register
```

How Run-Time Binding Works (Step-by-Step)

1. Program generates a **logical address**
2. Address is sent to the **MMU**
3. MMU:
   - Adds the **relocation register value**
4. Final **physical address** is sent to memory
5. Memory access happens

➡ This happens **for every memory reference**, during execution

---

Key Advantages

- Process can be **moved anywhere in memory**
- OS just updates the **relocation register**
- Program continues **without modification**

✔ This is **execution-time binding**

Important Points to Remember (Exam Gold ⭐)

- MMU performs **run-time address mapping**
- Binding happens **during execution**
- Logical → Physical mapping is **transparent to the user**
- Hardware support is **mandatory**
- Enables **process relocation & protection**

One-Line Summary

> **The MMU maps logical addresses to physical addresses at run time using a relocation register, allowing flexible and safe memory management.**

Quick check (answer in one line 👇):

Why do user programs **never see physical addresses**?

becuse of salt addedd by relocation register

You're **close**, but let's clean it up and make it **technically correct** 👍

Fixing Your Answer

You said:

> "because of salt added by relocation register"

The **idea is right**, but the wording needs correction.

Correct Version (Exam-Ready):

> **User programs never see physical addresses because the MMU adds the relocation (offset) value to logical addresses at run time, hiding physical memory locations from the program.**

Why “salt” is not the right word

- The correct term is **offset** or **relocation value**
- This value is stored in the **relocation (base) register**
- It is **automatically added** by the MMU

Simple Way to Remember 🧠

- Program says: “**Go to address 100**” (logical)
- MMU says: “**Actually go to 100 + base**” (physical)
- Program **never knows the final address**

One-Line Memory Trick

> **Logical addresses are translated by the MMU using a relocation offset, so physical addresses remain hidden from user programs.**

Nice effort — just replace *salt* with **offset / relocation value** and you’re perfect 👌

A process can be swapped out of memory to a backing store
then brought back into memory for continued execution
“total physical memory space of processes can exceed
total physical memory available”
Backing store – disk large enough to accommodate copies of memory images for all users
Major part of swap time is the transfer time
transfer time directly proportional to amount of memory swapped
Problem: Consider a process of size 10 MB. Disk to memory transfer rate is 5 MB per sec. Avg disk latency is 8

msec (no seek time). Calculate total cost of swapping. (for swap out + swap in)

Good — this is **swapping**. Let's understand it **clearly**, then we'll **work through the problem together** (without jumping to the final answer yet).

What is Swapping?

Swapping means:

- A process is **temporarily moved from main memory (RAM)** to a **backing store (disk)**
- Later, it is **brought back into memory** to continue execution

➡️ This allows:

Total memory required by processes > physical RAM available

Backing Store

- A **disk space** used to store swapped-out processes
- Must be large enough to hold **entire memory images** of processes
- Usually part of the **secondary storage**

Cost of Swapping

Swapping time mainly depends on:

1. **Transfer time**
   - Time to copy data between disk and memory
   - **Directly proportional to process size**
2. **Disk latency**
   - Delay before transfer starts
   - Given as **average latency**
   - (Seek time ignored here)

Given Problem (Let's Organize Data)

- Process size = **10 MB**
- Disk → Memory transfer rate = **5 MB/sec**
- Average disk latency = **8 ms**
- Swapping includes:
  - **Swap out**
  - **Swap in**

Step-by-Step Thinking (Your Turn)

Step 1️⃣

How long does it take to **transfer 10 MB once**, if the rate is **5 MB/sec**?

👉 *(Just calculate transfer time, ignore latency for now.)*



2.8 sec

Almost there — but let's **slow down and re-check the calculation together** 👇

Step 1: Transfer Time Formula

$$\text{Transfer Time} = \frac{\text{Amount of Data}}{\text{Transfer Rate}}$$

Plug in the values

- Amount of data = **10 MB**
- Transfer rate = **5 MB/sec**

So:

$$\frac{10}{5} = ?$$

👉 What do you get **now**?

(Just answer with the number — seconds.)



with latency'

Good catch 👍 — latency **does matter**, but let's add it **at the correct step**.

We'll do this **cleanly and slowly**.

Step 1️⃣ Transfer time (ONLY data transfer)

Formula:

$$\text{Transfer time} = \frac{\text{Data size}}{\text{Rate}}$$

$$= \frac{10\ \text{MB}}{5\ \text{MB/sec}} = 2\ \text{seconds}$$

✅ So **pure transfer time = 2 sec** (not 2.8)

Step 2️⃣ Add disk latency (for ONE operation)

- Average disk latency = **8 ms**
- Convert to seconds:

$$8\ \text{ms} = 0.008\ \text{sec}$$

Step 3️⃣ Time for ONE swap (either out OR in)

$$\text{One swap time} = 2 + 0.008 = 2.008\ \text{sec}$$

Step 4️⃣ Total cost of swapping (swap out + swap in)

Swapping happens **twice**:

- Swap out
- Swap in

$$\text{Total cost} = 2 \times 2.008 = 4.016 \text{ sec}$$

✅ Final Answer (Exam Style)

Total cost of swapping = 4.016 seconds

One-line memory tip 🧠

Swapping cost = 2 × (transfer time + latency)

If you want, next we can:

- Do a **similar numerical**
- Or compare **swapping vs paging**

Show how the available memory of 810 KB will be allocated using MVT approach, when job sequence is as given below; use all three placement algorithms.
Job Sequence:
J1 (90K), J2 (45K), J3 (180K), J4 (90K), J5 (135K), J6 (180K)
J3 terminates, J5 terminates
J7 (135K), J8 (180K)
J7 terminates, J8 terminates
J9 (285K)

This is a **classic OS exam problem** on **MVT (Multiple Variable Partitions)** with **First Fit, Best Fit, and Worst Fit**.

We'll do it **step by step**, and **you will fill the table with me** — I won't dump the full answer at once 👍

First: recall what MVT means (quick check)

In **MVT**:

- Memory is **not divided in advance**
- Each job gets **exactly the memory it needs**
- When a job terminates → it leaves a **hole**
- Placement algorithm decides **which hole to use**

✔ Correct so far? (just say *yes*)

Given Data

Total memory = 810 KB

Job sequence:

scss Copy code

```
J1 (90K), J2 (45K), J3 (180K), J4 (90K), J5 (135K), J6 (180K)
J3 terminates, J5 terminates
J7 (135K), J8 (180K)
J7 terminates, J8 terminates
J9 (285K)
```

Step 1️⃣ Initial Allocation (same for all algorithms)

At the **beginning**, memory is empty, so **First / Best / Worst Fit behave the same**.

Let's allocate **one job at a time from top of memory**.

Allocate:

- J1 = 90K
- J2 = 45K
- J3 = 180K
- J4 = 90K
- J5 = 135K
- J6 = 180K

👉 **Your turn (important):**

Question:
After allocating J1 to J6,
how much **free memory is left** out of 810 KB?

(Just calculate total used and subtract. Answer with a number.)

90k

✔️ **Correct — 90 KB free**
Good, let's keep going.

Memory state after J1–J6 allocation

Total memory = **810 KB**
Used = **720 KB**
Free = **90 KB**

Layout (top → bottom, conceptually):

mathematica Copy code

```
J1  90K
J2  45K
J3 180K
J4  90K
```

```
J5 135K
J6 180K
FREE 90K
```

Step 2️⃣ Termination of J3 and J5

Now:

- **J3 (180K) terminates**
- **J5 (135K) terminates**

These create **holes** (free blocks).

New holes:

- Hole A = **180K** (from J3)
- Hole B = **135K** (from J5)
- Hole C = **90K** (end of memory)

So free blocks are:

mathematica Copy code

```
180K, 135K, 90K
```

Important point (exam gold ⭐)

From **now on**, the behavior of:

- **First Fit**
- **Best Fit**
- **Worst Fit**

will be **different**.

Step 3️⃣ Allocate J7 (135K)

We now place **J7 = 135K**.

Question (answer ONE algorithm at a time):

🔹 **For First Fit:**

Which hole will J7 go into?

- 180K
- 135K
- 90K

👉 Tell me **which size hole** First Fit chooses.

125k

Good attempt 👍 — you're thinking in the **right direction**, but there's a **small size mistake** we need to fix before moving on.

Correction first (important)

There is **no 125 KB hole**.

The available holes are:

- **180 KB** (from J3)
- **135 KB** (from J5)
- **90 KB** (end of memory)

And **J7 = 135 KB**.

Now think about First Fit

First Fit rule:

> Allocate the job to the **first hole (from low memory to high memory)** that is **large enough**.

Memory order of holes:

1. **180 KB** ← first hole encountered
2. 135 KB
3. 90 KB

Question (one step only 👇):

Which hole will **First Fit** choose for **J7 (135 KB)**?

👉 Answer with the **hole size** (180K / 135K / 90K).

as j5 was 135k and is now empty hole and j7 is also 135k so it will go there

Excellent explanation 👏 — your **reasoning is correct**, but there's **one ordering detail** to fix (this is where exam answers often lose marks).

Key rule to remember

First Fit

> Chooses the **first hole in memory order** that is **large enough**,
> **not** the closest size match.

Memory order of holes (top → bottom)

After J3 and J5 terminate, memory looks like this:

mathematica Copy code

```
J1  90K
J2  45K
HOLE 180K   ← from J3
```

```
J4  90K
HOLE 135K   ← from J5
J6 180K
HOLE 90K
```

Applying First Fit to J7 (135K)

- First hole encountered = **180K**
- 180K ≥ 135K ✔

So:

> **First Fit allocates J7 into the 180 KB hole**

- Remaining hole = **45 KB** (180 − 135)

Final result for this step (First Fit)

- J7 → **180K hole**
- New hole created → **45K**

Quick check (answer briefly 👇):

Under **Best Fit**, which hole would J7 (135K) go into:

- 180K
- 135K
- 90K

135k

✔ **Correct!**

That's exactly right.

Why Best Fit chooses 135K

Best Fit rule

> Allocate the job to the **smallest hole that is large enough**.

Available holes:

- 180K
- **135K**
- 90K (too small)

➡️ **135K is the smallest hole that fits J7 (135K)**

➡️ **Perfect fit → no leftover hole**

Summary so far (important for final table)

Allocation of J7 (135K)

Algorithm	Hole chosen	Fragment left
First Fit	180K	45K
Best Fit	135K	0
Worst Fit	❓	❓

Next step (one question 👇):

Under **Worst Fit**, which hole will J7 go into?

(Options: 180K / 135K / 90K)

first do with best fit , solve completely

Got it 👍 — we'll do **ONLY BEST FIT**, **cleanly and completely**, exactly the way it should appear in an **OS exam answer**.

I'll still **explain each step briefly**, so you can reproduce it yourself.

MVT Allocation using BEST FIT

Total Memory = 810 KB

Step 1️⃣ Initial Job Allocation (J1 to J6)

At the start, memory is empty → Best Fit behaves like normal allocation.

Job	Size (KB)	Status
J1	90	Allocated
J2	45	Allocated
J3	180	Allocated
J4	90	Allocated
J5	135	Allocated
J6	180	Allocated

Total used = 720 KB

Free memory = 90 KB

Step 2️⃣ Termination of J3 and J5

- J3 (180 KB) terminates → hole of **180 KB**
- J5 (135 KB) terminates → hole of **135 KB**

Free holes now:

- 180 KB
- 135 KB
- 90 KB

Step 3️⃣ Allocate J7 (135 KB) — Best Fit

Best Fit chooses the **smallest hole that can fit the job**.

- 90 KB ❌ (too small)
- **135 KB ✅ (exact fit)**
- 180 KB (bigger, not best)

➡️ **J7 is allocated to the 135 KB hole**
➡️ **No fragmentation**

Step 4️⃣ Allocate J8 (180 KB) — Best Fit

Available holes:

- 180 KB
- 90 KB

Best Fit choice:

- **180 KB (exact fit)**

➡️ **J8 allocated to 180 KB hole**
➡️ **No fragmentation**

Step 5️⃣ Termination of J7 and J8

- J7 (135 KB) terminates → hole **135 KB**
- J8 (180 KB) terminates → hole **180 KB**

Free holes now:

- 180 KB
- 135 KB
- 90 KB

(Same pattern again)

Step 6️⃣ Allocate J9 (285 KB) — Best Fit

Available holes:

- 180 KB
- 135 KB
- 90 KB

👉 Largest hole = 180 KB
👉 **285 KB cannot fit in any hole**

❌ Final Result (Best Fit)

> **J9 (285 KB) cannot be allocated due to external fragmentation**,
> even though total free memory = **405 KB**

Final Conclusion (Exam-Ready)

- Best Fit minimizes fragmentation for smaller jobs
- But creates **many small holes**
- Large jobs may fail due to **external fragmentation**
- **J9 cannot be allocated under Best Fit**

If you want, next we can:

- Do **First Fit** completely
- Or **Worst Fit**
- Or write the **final comparison conclusion** (very common exam question)



no go with first fit

Perfect 👍 — now we'll do **ONLY FIRST FIT**, **step by step**, exactly how you should write it in an exam.

MVT Allocation using FIRST FIT

Total Memory = 810 KB

Step 1️⃣ Initial Allocation (J1 to J6)

At the beginning, memory is empty, so jobs are placed sequentially.

Job	Size (KB)	Status
J1	90	Allocated
J2	45	Allocated
J3	180	Allocated
J4	90	Allocated
J5	135	Allocated
J6	180	Allocated

Total used = 720 KB

Free memory = 90 KB

Step 2️⃣ Termination of J3 and J5

- J3 (180 KB) terminates → creates **180 KB hole**
- J5 (135 KB) terminates → creates **135 KB hole**

Holes in memory order:

Copy code

```
180 KB, 135 KB, 90 KB
```

Step 3️⃣ Allocate J7 (135 KB) — First Fit

First Fit rule:

> Allocate the job in the **first hole (in memory order)** that is large enough.

- First hole = **180 KB** ✔ fits J7

➡️ **J7 is allocated in the 180 KB hole**

Fragment left:

- 180 − 135 = **45 KB**

Holes now:

Copy code

```
45 KB, 135 KB, 90 KB
```

Step 4️⃣ Allocate J8 (180 KB) — First Fit

Check holes in order:

- 45 KB ❌
- 135 KB ❌
- 90 KB ❌

➡️ **No hole large enough**

Result:

> **J8 cannot be allocated using First Fit**

Step 5️⃣ Termination of J7

- J7 (135 KB) terminates

- The **45 KB fragment + 135 KB block merge**

➡️ New hole = **180 KB**

Holes now:

Copy code

```
180 KB, 90 KB
```

Step 6️⃣ Allocate J9 (285 KB) — First Fit

Check holes:

- 180 KB ❌
- 90 KB ❌

➡️ **J9 cannot be allocated**

✅ Final Result (First Fit)

- J7 allocated successfully
- J8 **not allocated**
- J9 **not allocated**
- External fragmentation present

One-line Exam Conclusion 🧠

> **In First Fit MVT, large jobs may fail due to external fragmentation even when total free memory is sufficient.**

If you want, next we can:

- Do **Worst Fit**
- Or write a **comparison table (First vs Best vs Worst)**
- Or practice drawing the **memory layout diagram** (very common in exams)

do worst fit

Alright 👍 — now we'll do **ONLY WORST FIT**, **step by step**, clean and **exam-ready**, just like the previous two.

MVT Allocation using WORST FIT

Total Memory = 810 KB

Step 1️⃣ Initial Allocation (J1 to J6)

At the start, memory is empty, so allocation is straightforward.

Job	Size (KB)	Status
J1	90	Allocated
J2	45	Allocated
J3	180	Allocated
J4	90	Allocated
J5	135	Allocated
J6	180	Allocated

Total used = 720 KB

Free memory = 90 KB

Step 2️⃣ Termination of J3 and J5

- J3 (180 KB) terminates → hole of **180 KB**
- J5 (135 KB) terminates → hole of **135 KB**

Free holes now:

Copy code

```
180 KB, 135 KB, 90 KB
```

Step 3️⃣ Allocate J7 (135 KB) — Worst Fit

Worst Fit rule:

> Allocate the job to the **largest available hole**.

Largest hole = **180 KB**

➡️ **J7 is allocated to the 180 KB hole**

Fragment left:

- 180 − 135 = **45 KB**

Holes now:

Copy code

```
45 KB, 135 KB, 90 KB
```

Step 4️⃣ Allocate J8 (180 KB) — Worst Fit

Check largest hole:

- Largest = **135 KB**

But:

- 135 KB < 180 KB ❌

➡️ **J8 cannot be allocated**

Step 5️⃣ Termination of J7

- J7 (135 KB) terminates

- The **45 KB fragment + 135 KB block merge**

➡️ New hole = **180 KB**

Holes now:

Copy code

```
180 KB, 90 KB
```

Step 6️⃣ Allocate J9 (285 KB) — Worst Fit

Check largest hole:

- 180 KB < 285 KB ❌

➡️ **J9 cannot be allocated**

✅ Final Result (Worst Fit)

- J7 allocated
- J8 **not allocated**
- J9 **not allocated**
- Large fragmentation remains unused

Final Comparison Insight (Very Important for Exams ⭐)

Algorithm	J7	J8	J9
First Fit	✔	❌	❌
Best Fit	✔	✔	❌
Worst Fit	✔	❌	❌

One-line Conclusion 🧠

> **Worst Fit leaves large holes initially but still suffers from external fragmentation, preventing large job allocation.**

If you want next:

- I can help you **draw neat memory diagrams**
- Or give a **model exam answer**
- Or help you **compare algorithms in 5 lines**

what is internal and external fragmentation

Good question — this is a **core OS concept** and often asked directly in exams.
Let's keep it **clear, short, and exam-ready**.

🔹 Internal Fragmentation

Definition

Internal fragmentation occurs when:

> **Allocated memory is larger than the memory requested**, and the **unused space lies inside the allocated block**.

Why it happens

- Memory is allocated in **fixed-size blocks**
- A process may not use the entire block

Example

- Block size = 100 KB
- Process needs = 90 KB

➡️ **10 KB wasted inside the block**
➡️ This wasted space **cannot be used by other processes**

Where it occurs

- Fixed partitioning
- Paging

🔹 External Fragmentation

Definition

External fragmentation occurs when:

> **Enough total free memory exists, but it is split into small non-contiguous holes**, so a large process cannot be allocated.

Why it happens

- Variable-sized allocation
- Processes enter and leave memory dynamically

Example

Free holes:

Copy code

```
50 KB + 40 KB + 60 KB = 150 KB (total)
```

Process needs:

Copy code

```
100 KB
```

➡️ Cannot be allocated because no **single contiguous block** is large enough

Where it occurs

- MVT (variable partitioning)
- Segmentation

🔑 Key Differences (Exam Table)

Internal Fragmentation	External Fragmentation
Waste **inside** allocated block	Waste **between** allocated blocks
Caused by fixed block size	Caused by variable block size
Unused space cannot be reused	Total free space exists but scattered
Occurs in paging	Occurs in MVT / segmentation

One-Line Memory Trick 🧠

> **Internal = waste inside**
> **External = waste outside**

Quick check (1 line 👇):

Which fragmentation occurs in **paging**, and why?

What is pagging

Good — this is a **basic but very important OS concept**.
Let's understand **paging** step by step.

What is Paging?

Paging is a **memory management technique** in which:

> **Logical memory is divided into fixed-size pages, and physical memory is divided into fixed-size frames.**

A process is loaded by placing its pages into **any available frames**, **not necessarily contiguous**.

Key Terms (Must know)

- **Page** → Fixed-size block of a process (logical memory)
- **Frame** → Fixed-size block of physical memory
- **Page size = Frame size**

How Paging Works (Simple Flow)

1. Process is divided into **pages**
2. Main memory is divided into **frames**
3. Pages are placed into **any free frames**
4. **Page table** keeps track of:
   - Page number → Frame number
5. CPU uses the **MMU** to translate addresses

Why Paging is Used

✔ Eliminates **external fragmentation**
✔ No need for contiguous memory
✔ Efficient memory utilization

⚠ May cause **internal fragmentation**

Address Structure in Paging

A logical address is split into:

mathematica Copy code

```
| Page Number | Page Offset |
```

MMU uses:

- Page number → find frame number
- Offset → exact location inside the frame

Simple Example

- Page size = 4 KB
- Process size = 10 KB

Pages:

- Page 0 → 4 KB
- Page 1 → 4 KB
- Page 2 → 2 KB

➡️ Last page wastes **2 KB** → internal fragmentation

One-Line Exam Definition ✅

> **Paging is a memory management scheme that divides processes into fixed-size pages and maps them to fixed-size frames in physical memory, eliminating external fragmentation.**

Quick check (answer in one line 👇):

Does paging cause **internal fragmentation** or **external fragmentation**, and why?